

## Ch1 初识C语言与程序框架 · 练习

对应《C Primer Plus（第6版）》第 1-2 章 · 参考答案含「思路 + 常见错法」· 请先独立完成再看答案

### 1-1 填空

**填空** ① 每个 C 程序从 \_\_\_\_\_ 函数开始执行；用 printf 需要先 #include <\_\_\_\_\_>。

### 1-2 填空

**填空** ② 声明一个装整数的变量 num 并初始化为 7，写为 \_\_\_\_\_ = \_\_\_\_\_；

### 1-3 填空

**填空** ③ return 0；表示程序 \_\_\_\_\_；返回非 0 通常表示 \_\_\_\_\_。

### 1-4 改错

**改错** ④ 下面程序有 3 处错误，先自己找，再对照答案。

```
/* debug_exercise_101.c - 找 3 处错误 */
#include <stdio.h>

int main(void)
{
    int age
    printf("我 %d 岁\n" age);
    return 0;
}
```

### 1-5 编程

**编程** ⑤ 写一个程序，用三个变量（年龄、身高、姓名）打印一行自我介绍。示例输出：

```
ch1_p5_profile.c
我 19 岁，身高 175，姓名 F-VIS
```

## 1-6 编程

**编程** ⑥ 写一个程序，定义两个整数  $a=17$ 、 $b=25$ ，计算并打印  $a+b$  的和。示例输出：

```
ch1_p6_add.c  
17 + 25 = 42
```

## 1-7 专业场景

**专业场景** ⑦（不需要硬件知识）嵌入式里常要模拟"逐个点亮一组 LED"。写一个程序，用循环从 1 数到 8，每行打印 LED\_1=ON 到 LED\_8=ON。提示：只需要一个循环和一个 printf。

## 1-8 索引

**索引** ⑧ 打开《C Primer Plus（第6版）》第 1、2 章，完成 复习题 1、2、3（题号对应主教材）。

## 参考答案

① `main; stdio.h。`

**思路：**main 是入口，没有它链接会报 `undefined reference to 'main'`；printf 在 `stdio.h` 里声明。

② `int num = 7;`

**思路：**声明时顺手初始化，避免"残留值"。常见错法：写 `int num = 7` 漏分号。

③ 正常结束；出错。

**思路：**0 是约定俗成的"成功"。shell 里可以看 `echo $?` 拿到返回值。

#### ④ 三处错误：

1. 第 6 行 `int age` **漏分号**——报错会落在第 7 行附近（见讲义 1.4 对照表第一行）；
2. 第 7 行 `printf("我 %d 岁\n" age)`——**字符串和参数之间漏逗号**，应为 `printf("我 %d 岁\n", age)`；
3. `age` **声明了未初始化就用**——这是**第三处**，但前两处是硬错误，**gcc 没跑到这一处**；把前两处修掉后 `gcc -Wall -Wextra` 会报 `warning: 'age' is used uninitialized [-Wuninitialized]`。整段含硬错误时看不到后面的警告（口径同 Ch10 练习册 ①）。

**诊断口径：**漏分号 + 漏逗号都属于"语法错误"，gcc 会 `error` 且编译失败；未初始化属"警告"（`warning`），编译能过但结果不确定。**修完三处后请实跑确认输出。**

#### ⑤ 参考代码：

```
ch1_p5_profile.c
#include <stdio.h>

int main(void)
{
    int    age    = 19;
    double height = 175.0;
    char   name[] = "F-VIS";

    printf("我 %d 岁，身高 %g，姓名 %s\n", age, height, name);
    return 0;
}
```

**思路：**小数用 `double + %g`（自动去尾零）打印更干净。**常见错法：**给 `double` 用 `%d`——输出乱码。

#### ⑥ 参考代码：

```
ch1_p6_add.c
#include <stdio.h>

int main(void)
{
    int a = 17;
    int b = 25;
    printf("%d + %d = %d\n", a, b, a + b);
    return 0;
}
```

**思路：**计算可以直接写在 printf 参数里。**常见错法：**三个 %d 但只给两个参数——gcc 会警告 format 参数不足。

#### ⑦ 参考代码（LED 模拟）：

```
ch1_p7_led.c
#include <stdio.h>

int main(void)
{
    int i;
    for (i = 1; i <= 8; i++) {
        printf("LED_%d=ON\n", i);
    }
    return 0;
}
```

**思路：**用 for 循环 + %d 把编号变起来。这是"模拟硬件行为"的第一课：真实嵌入式里 LED\_x=ON 会变成"把某个寄存器/引脚置 1"。